Weather Prediction Analytics

An end-to-end ELT pipeline with Snowflake, Apache Airflow, dbt and Apache Superset

Team	Rahil Shah · Student ID 020755137 Dev Faldu · Student ID 020792174
Course	DATA 226 — San José State University
GitHub repository	https://github.com/rahil88/weather-analytics-lab
Stack	Open-Meteo API · Airflow 2.10.1 · Snowflake · dbt 1.8 · Superset 4.1.1
Date	September 29, 2026

Contents

1. Team Introduction
2. Problem Statement
3. Requirements & Specifications
4. Dataset(s)
5. System Diagram
6. Tables
7. Implementation Details — Airflow, dbt, Superset
8. Lessons
9. Future Work
10. Conclusion
11. References

1. Team Introduction

Name	Student ID	Contact / Repository
Rahil Shah	020755137	GitHub: github.com/rahil88/weather-analytics-lab
Dev Faldu	020792174	

We built a production-style weather analytics platform. It collects daily observations and 7-day forecasts for two US cities, loads them into a cloud data warehouse, transforms them into climate metrics with tested SQL, and presents them in an interactive dashboard. The pipeline runs every day without manual steps.

2. Problem Statement

Raw daily weather values, such as a maximum temperature or a precipitation total, say what happened on one day. They do not answer the questions people actually plan around:

- Is this week **warmer or colder than normal** for this city?
- What is the **trend** once day-to-day noise is smoothed out?
- How much rain has fallen **recently**? Has a **dry spell** started, and how long has it lasted?
- How do two very different climates, dry Mediterranean **San Jose** and wet marine **Seattle**, compare?
- What does the **7-day forecast** mean relative to that city's baseline?

Our goal was to automate data collection, store the data reliably and idempotently in Snowflake, compute these metrics with version-controlled and tested dbt models, and present them in a BI dashboard that refreshes daily.

3. Requirements & Specifications

3.1 Requirements

#	Requirement	How it is met
R1	Weather for two or more cities from Open-Meteo	`weather_etl` fetches every city in the `weather_cities` Airflow Variable (San Jose, Seattle)
R2	Raw data stored in Snowflake	`DEMO_DB.RAW.WEATHER_DAILY`, primary key `(city, date)`
R3	Airflow Connections and Variables; no secrets in code	Connection `snowflake_conn` (key-pair); Variables `weather_cities`, `weather_past_days`, `weather_forecast_days`
R4	Idempotent load: SQL transaction + try/except/raise	`BEGIN → DELETE window → INSERT → COMMIT`; `ROLLBACK` + `raise` on error
R5	dbt models, tests and snapshot	3 models, 23 data tests, 1 SCD-2 snapshot

R6	dbt scheduled as an Airflow DAG after the ETL	`weather_dbt` runs on `schedule=[Dataset]`, which the ETL's `load` task updates
R7	BI dashboard of the dbt metrics	Superset dashboard with 6 charts and City + Date Range filters

3.2 Specifications

Layer	Technology	Version / setting
Source	Open-Meteo Forecast API `/v1/forecast` (no key)	60 days history + 7 days forecast per run
Orchestration	Apache Airflow in Docker (LocalExecutor)	2.10.1, UI on :8081
Warehouse	Snowflake	Warehouse `DEMO_WH` (XS, auto-suspend 60 s), DB `DEMO_DB`
Transformation	dbt-core + dbt-snowflake	1.8.7 / 1.8.1
BI	Apache Superset in Docker	4.1.1, UI on :8088
Authentication	RSA key-pair (private key mounted read-only, never committed)	Used by Airflow, dbt and Superset

4. Dataset(s)

Source: Open-Meteo Forecast API, `https://api.open-meteo.com/v1/forecast`, a free, key-less JSON API that serves ECMWF/GFS/HRRR-based weather models.

City	Latitude	Longitude	Timezone	Climate
San Jose, CA	37.3382	–121.8863	America/Los_Angeles	Mediterranean, dry summers
Seattle, WA	47.6062	–122.3321	America/Los_Angeles	Marine, wet autumn/winter

Request parameters (per city): `daily=temperature_2m_max, temperature_2m_min, temperature_2m_mean, precipitation_sum, rain_sum, wind_speed_10m_max, weather_code`, `past_days=60`, `forecast_days=7`, `timezone=America/Los_Angeles`. The API returns one array per field. The ETL turns these arrays into one row per city per day.

API field	Unit	Loaded as
temperature_2m_max / min / mean	°C	`temp_max`, `temp_min`, `temp_mean`
precipitation_sum / rain_sum	mm	`precipitation_sum`, `rain_sum`
wind_speed_10m_max	km/h	`wind_speed_max`
weather_code	WMO code	`weather_code` → mapped to `weather_category` in dbt
(derived)	boolean	`is_forecast` = date ≥ today in the city's timezone

Volume at time of writing: 71 days × 2 cities = 142 rows in `RAW.WEATHER_DAILY` (2026-07-27 → 2026-10-05), with 64 observed days and 7 forecast days per city. Because the load only replaces the window it fetched, history keeps growing by one day per city per day.

5. System Diagram

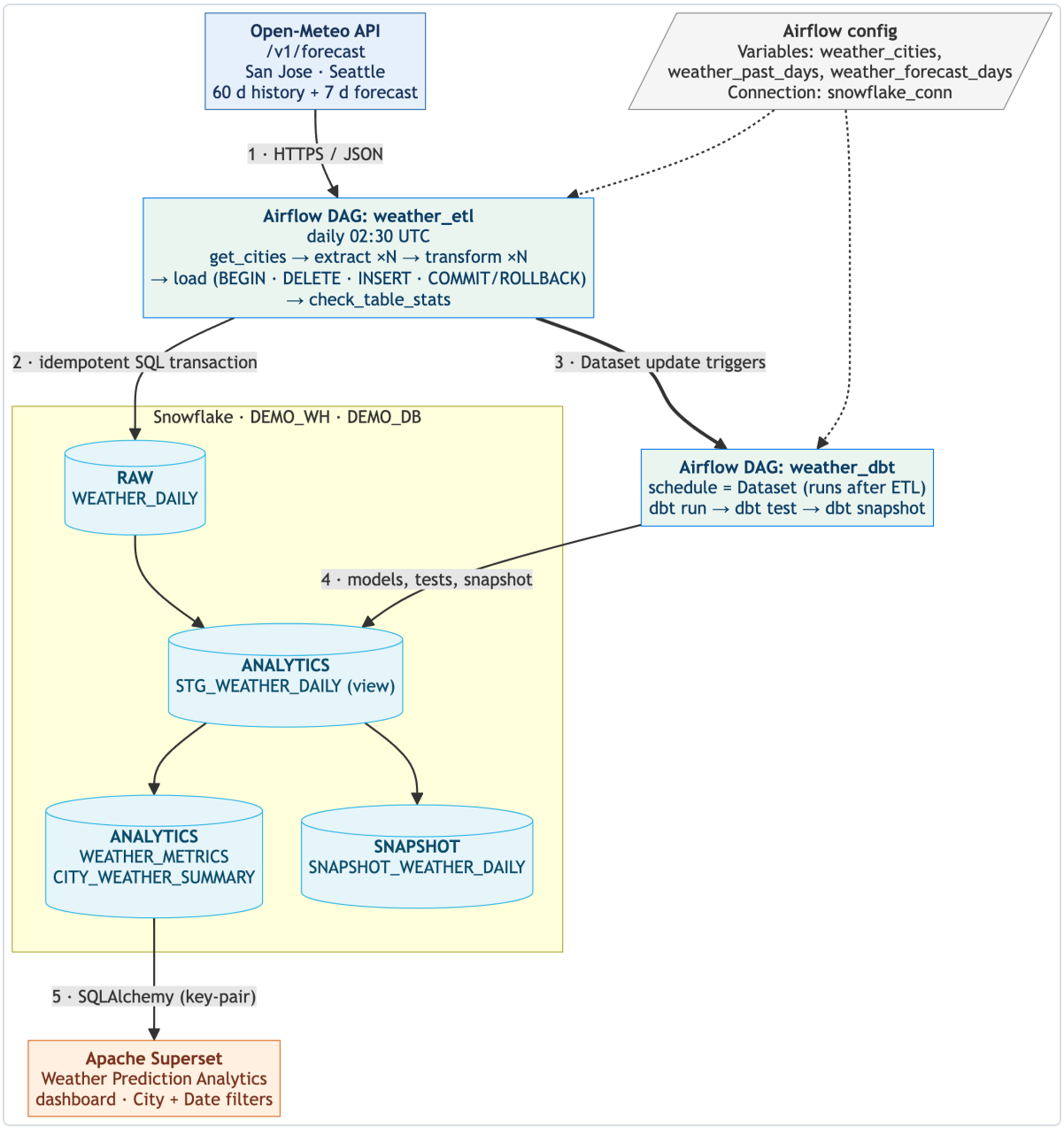


Figure 1 — Components and data flow (numbers show the order of each run)

1. **Extract:** `weather_etl` reads the city list from a Variable and runs one `extract` task per city (dynamic task mapping).
2. **Transform + Load:** each city's JSON is flattened into rows, and all cities are written in one Snowflake transaction.
3. **Trigger:** the `load` task updates the Dataset `snowflake://DEMO_DB/RAW/WEATHER_DAILY`, which triggers `weather_dbt`.
4. **dbt:** `dbt run` → `dbt test` → `dbt snapshot` build the `ANALYTICS` and `SNAPSHOT` schemas.

5. **BI:** Superset queries the `ANALYTICS` marts in Snowflake.

6. Tables

All objects live in `DEMO_DB`. Types and nullability are taken from Snowflake `INFORMATION_SCHEMA.COLUMNS`.

6.1 RAW.WEATHER_DAILY — table (created and loaded by Airflow)

Field	Type	Null	Constraint / default	Description
CITY	VARCHAR(50)	NO	**PK (1)**	City name
LATITUDE	NUMBER(9,6)	NO		Latitude
LONGITUDE	NUMBER(9,6)	NO		Longitude
DATE	DATE	NO	**PK (2)**	Local calendar day
TEMP_MAX / TEMP_MIN / TEMP_MEAN	FLOAT	YES		°C at 2 m
PRECIPITATION_SUM	FLOAT	YES		mm
RAIN_SUM	FLOAT	YES		mm (rain only)
WIND_SPEED_MAX	FLOAT	YES		km/h at 10 m
WEATHER_CODE	NUMBER(38,0)	YES		WMO code
IS_FORECAST	BOOLEAN	NO		TRUE for today and later
LOADED_AT	TIMESTAMP_NTZ	NO	DEFAULT CURRENT_TIMESTAMP()	Load time (dbt freshness field)

6.2 ANALYTICS.STG_WEATHER_DAILY — dbt view

Field	Type	Null	dbt tests	Description
WEATHER_ID	VARCHAR	YES	unique, not_null	Surrogate key `city|date`
CITY / DATE	VARCHAR(50) / DATE	NO	not_null	
LATITUDE / LONGITUDE	NUMBER(9,6)	NO		
TEMP_MAX_C / TEMP_MIN_C	FLOAT	YES	singular: max ≥ min	°C
TEMP_MEAN_C	FLOAT	YES	not_null	Falls back to (max+min)/2
TEMP_RANGE_C	FLOAT	YES		max – min
PRECIPITATION_MM / RAIN_MM	FLOAT	YES	not_null (precip.)	NULL → 0
WIND_SPEED_MAX_KMH	FLOAT	YES		
WEATHER_CODE	NUMBER(38,0)	YES		
WEATHER_CATEGORY	VARCHAR(12)	YES	accepted_values (10)	Clear, Cloudy, Fog, Drizzle, Rain, …

IS_FORECAST	BOOLEAN	NO	not_null	
LOADED_AT	TIMESTAMP_NTZ	NO		

6.3 ANALYTICS.WEATHER_METRICS — dbt table

Field	Type	dbt tests	Definition
WEATHER_ID	VARCHAR	unique, not_null, relationships→stg	Key
CITY, DATE, IS_FORECAST, WEATHER_CATEGORY	VARCHAR / DATE / BOOLEAN / VARCHAR		From staging
TEMP_MAX_C, TEMP_MIN_C, TEMP_MEAN_C, TEMP_RANGE_C, PRECIPITATION_MM, WIND_SPEED_MAX_KMH	FLOAT		From staging
TEMP_MA_7D_C / TEMP_MA_30D_C	FLOAT	not_null	Moving average of mean temp over last 7 / 30 days
BASELINE_TEMP_C	FLOAT		Mean of observed (non-forecast) days per city
TEMP_ANOMALY_C	FLOAT	not_null	temp_mean_c – baseline
TEMP_ANOMALY_ZSCORE	FLOAT		Anomaly ÷ stddev of observed days
RAIN_ROLLING_7D_MM / 30D_MM	FLOAT	not_null	Precipitation summed over last 7 / 30 days
IS_DRY_DAY	BOOLEAN	singular: consistency	precipitation < 1.0 mm (WMO)
DRY_SPELL_LENGTH_DAYS	NUMBER(13,0)	not_null, singular	Consecutive dry days ending on this date (0 if wet)

6.4 ANALYTICS.CITY_WEATHER_SUMMARY — dbt table (one row per city, observed days)

Field	Type	dbt tests	Description
CITY	VARCHAR(50)	unique, not_null	
FIRST_DATE / LAST_DATE	DATE		Observed range
OBSERVED_DAYS	NUMBER(18,0)		
AVG_TEMP_C / RECORD_HIGH_C / RECORD_LOW_C	FLOAT		°C
TOTAL_PRECIPITATION_MM	FLOAT		mm
DRY_DAYS / LONGEST_DRY_SPELL_DAYS	NUMBER(13,0)		
MAX_ABS_TEMP_ANOMALY_C	FLOAT		Largest deviation from baseline

6.5 SNAPSHOT.SNAPSHOT_WEATHER_DAILY — dbt snapshot (SCD Type 2)

All `STG_WEATHER_DAILY` columns, plus the dbt metadata columns below. Strategy `check` on `temp_max_c, temp_min_c, temp_mean_c, precipitation_mm, weather_code, is_forecast`, unique key `weather_id`.

Field	Type	Description
DBT_SCD_ID	VARCHAR(32)	Unique id of each row version
DBT_UPDATED_AT	TIMESTAMP_NTZ	When dbt captured the version
DBT_VALID_FROM / DBT_VALID_TO	TIMESTAMP_NTZ	Validity window (`DBT_VALID_TO` NULL = current)

7. Implementation Details

Repository layout: DAGs in `dags/`, the dbt project in `dbt/`, the Superset bootstrap script in `superset/`, and diagrams and screenshots in `docs/`. `docker-compose.yaml` starts Airflow, its Postgres metadata DB and Superset.

7.1 Airflow — DAGs

DAG	Schedule	Tasks
`weather_etl`	`30 2 * * *` (daily)	get_cities → extract [mapped per city] → transform [mapped] → load → check_table_stats
`weather_dbt`	`[Dataset("snowflake://DEMO_DB/RAW/WEATHER_DAILY")]`	dbt_run → dbt_test → dbt_snapshot

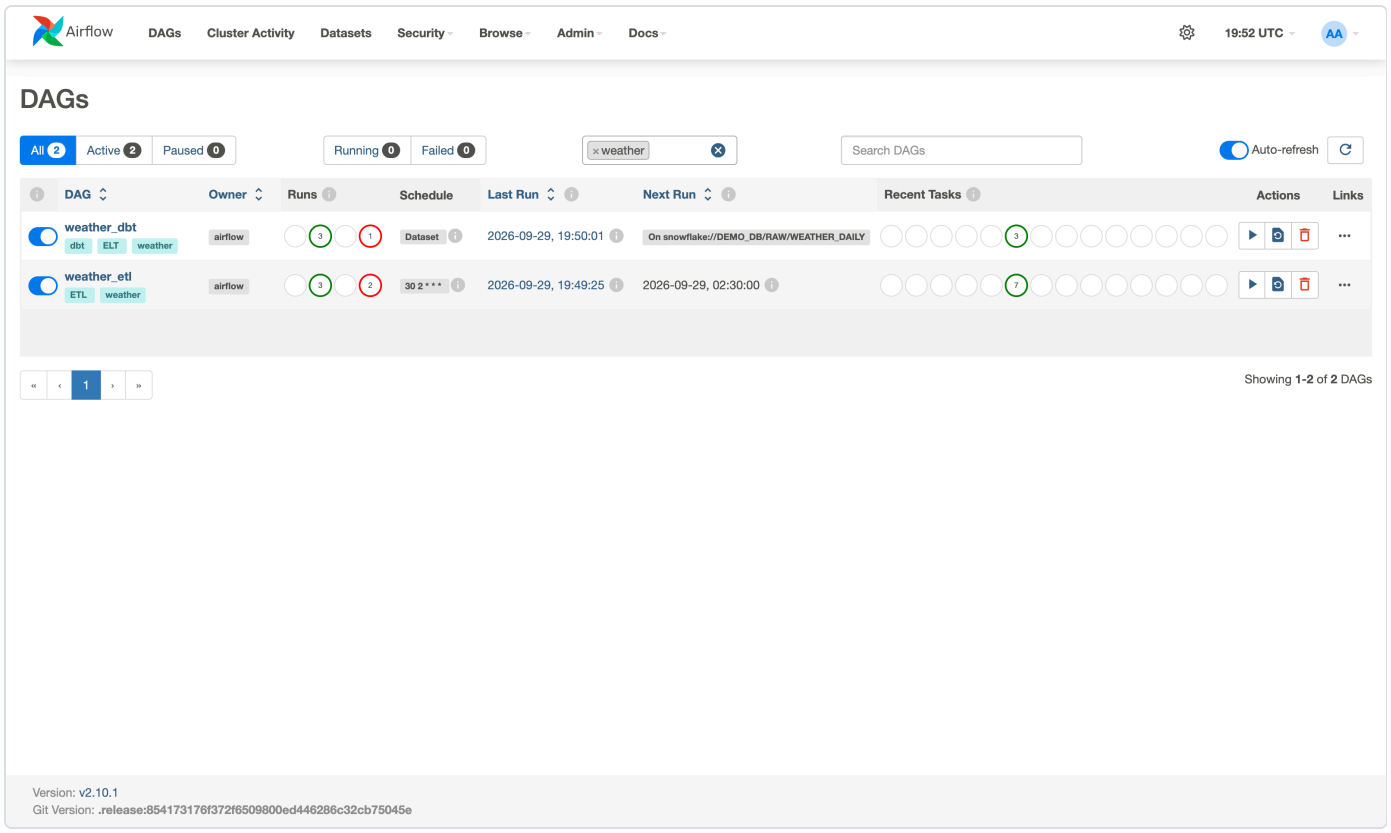


Figure 2 — Airflow DAG list: weather_etl (cron) and weather_dbt (Dataset-scheduled)

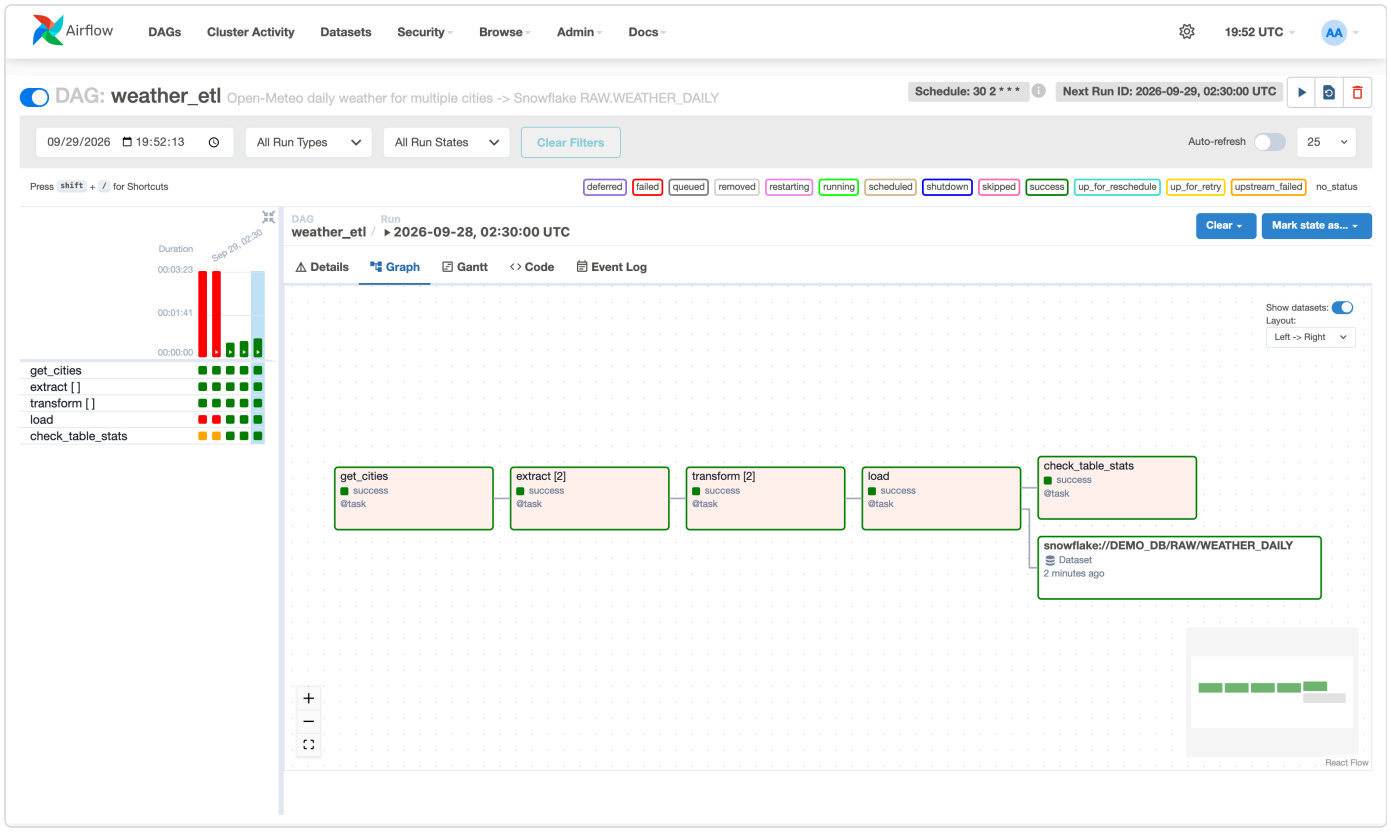


Figure 3 — ETL pipeline graph: extract [2] / transform [2] are mapped per city; load publishes the Dataset

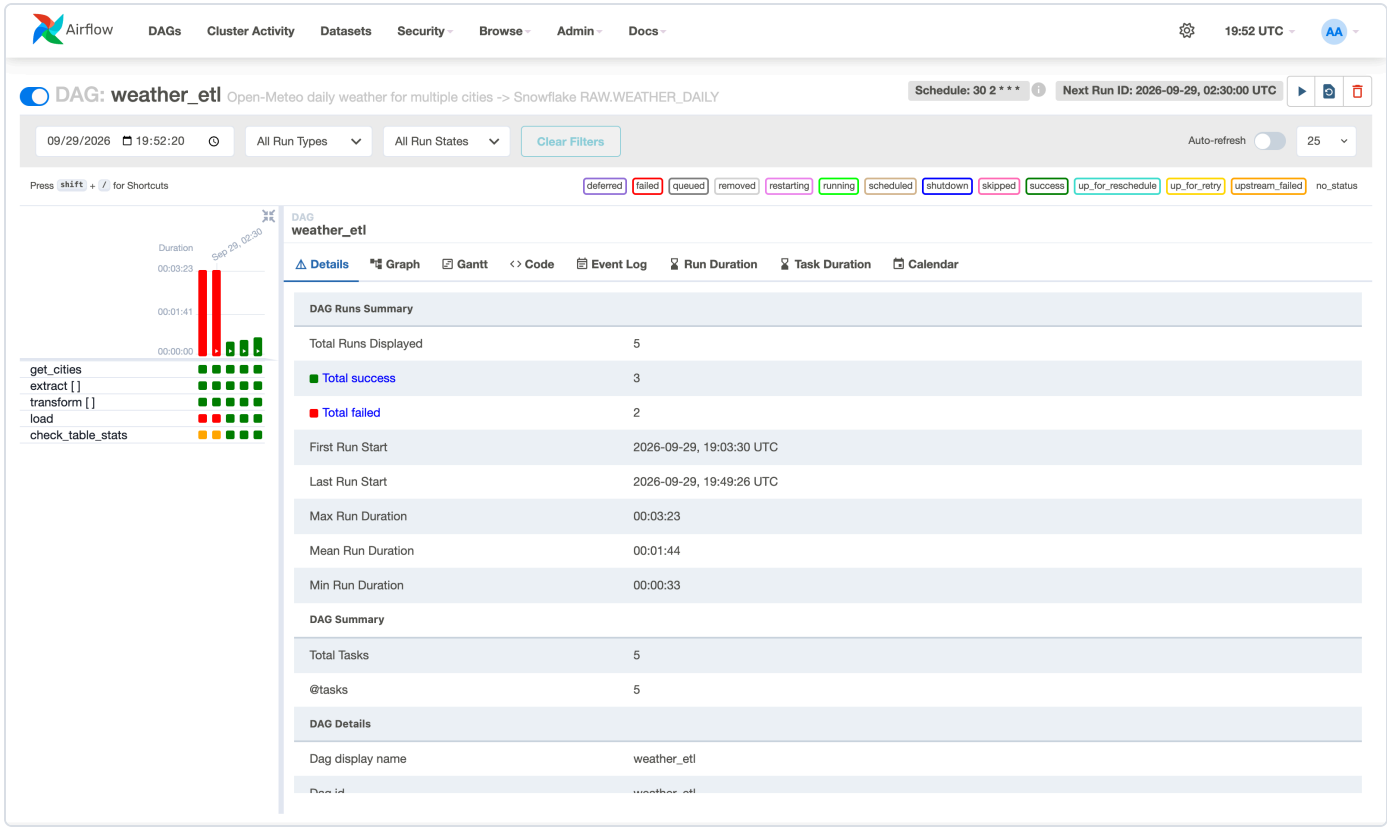


Figure 4 — ETL run history. The first two runs failed because the original Snowflake trial account had been suspended; every run after we moved to the new account succeeded.

7.2 Airflow — Connections and Variables

Kind	Key	Purpose
Connection	`snowflake_conn`	Type snowflake; login = Snowflake user; extras: account, warehouse DEMO_WH, database DEMO_DB, role, `private_key_file`
Variable	`weather_cities`	JSON list of {city, latitude, longitude, timezone}. Adding a city needs no code change
Variable	`weather_past_days`	Days of history per run (60)
Variable	`weather_forecast_days`	Days of forecast per run (7)

`weather_etl` connects via `SnowflakeHook(snowflake_conn_id="snowflake_conn")`. `weather_dbt` reads the same connection with `BaseHook.get_connection` and passes its fields to dbt as environment variables, so `dbt/profiles.yml` contains only `env_var()` look-ups and no credentials.

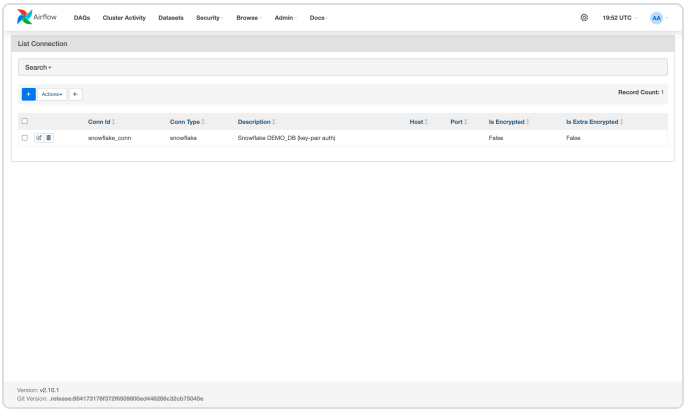


Figure 5 — Airflow Connection

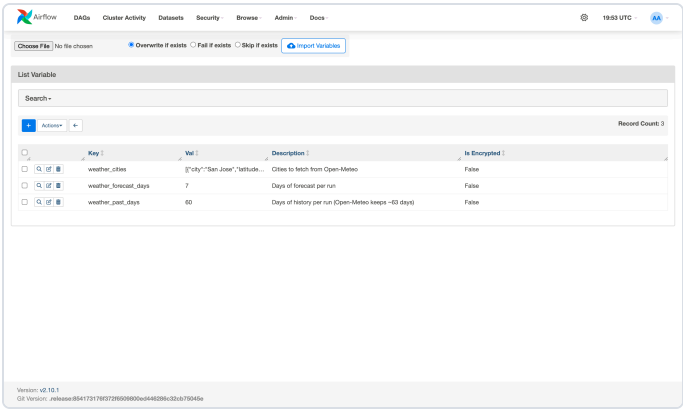


Figure 6 — Airflow Variables

7.3 Airflow — Idempotent load with an SQL transaction and try/except/raise

```
try:
    cur.execute("BEGIN;")
    for batch in city_batches:              # one batch per city (mapped task output)
        city, start, end, records = batch["city"], batch["start"], batch["end"], batch["rows"]
        cur.execute(
            f"DELETE FROM {target_table} WHERE city = %s AND date BETWEEN %s AND %s",
            (city, start, end),
        )
        cur.executemany(insert_sql, records)
    cur.execute("COMMIT;")
except Exception as e:
    cur.execute("ROLLBACK;")
    print(f"Load failed, transaction rolled back: {e}")
    raise
finally:
    cur.close()
```

- **Idempotent:** the whole requested window is deleted and re-inserted, so re-running (or retrying) produces the same table. After four runs the table still has exactly one row per (city, date), which the dbt `unique` test confirms.

- **Atomic:** all cities commit together. A failure part-way rolls everything back, and `raise` fails the task so Airflow retries it.
- `CREATE TABLE IF NOT EXISTS` runs before `BEGIN`, because DDL auto-commits in Snowflake.

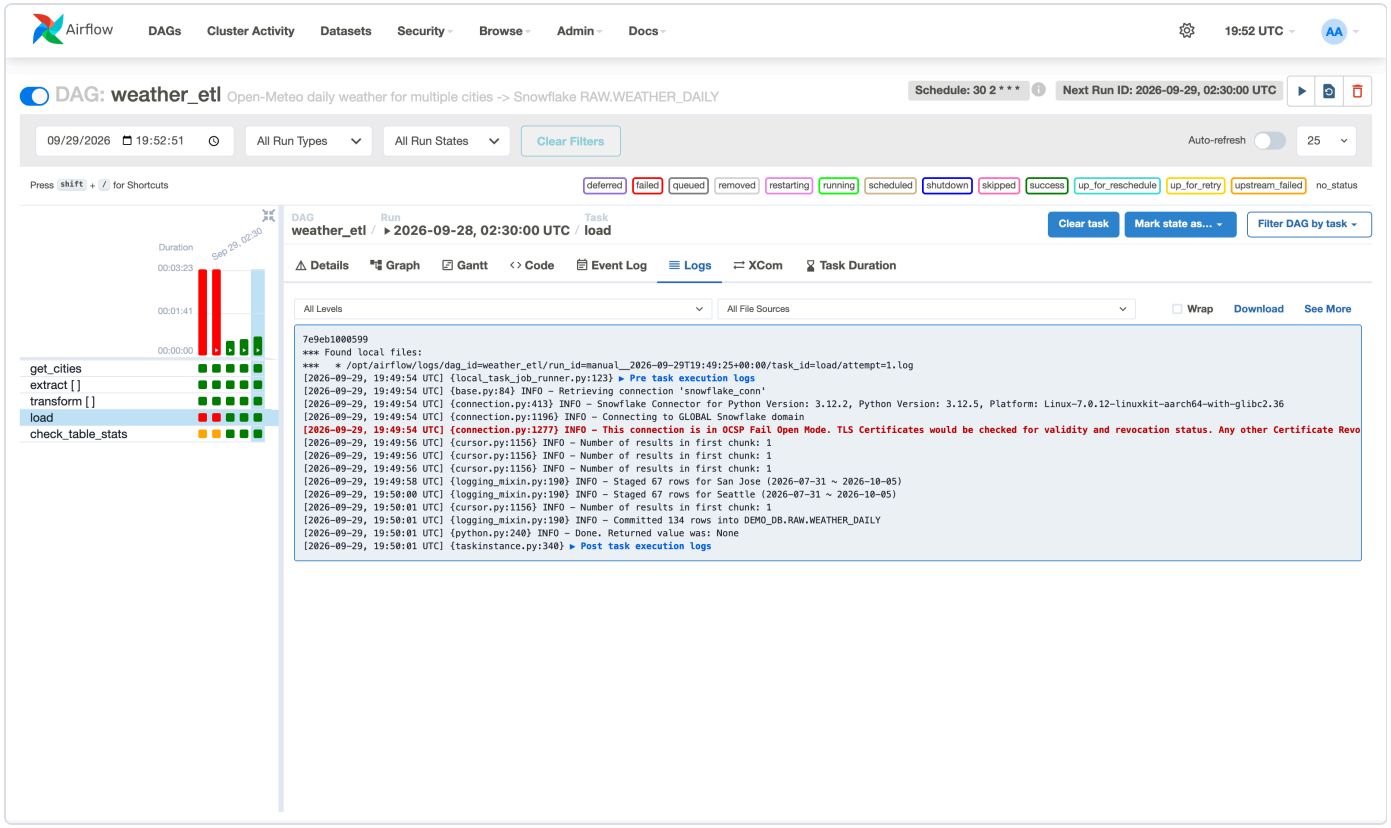


Figure 7 — load task log: rows staged per city, then a single commit of 134 rows

7.4 Airflow — dbt DAG scheduled after the ETL

```
WEATHER_DAILY_DATASET = Dataset("snowflake://DEMO_DB/RAW/WEATHER_DAILY")

@task(outlets=[WEATHER_DAILY_DATASET])      # weather_etl.load: marks the Dataset updated on success
def load(city_batches, target_table): ...

with DAG(dag_id="weather_dbt", schedule=[WEATHER_DAILY_DATASET], ...):   # consumer
    dbt_run >> dbt_test >> dbt_snapshot       # BashOperators: dbt <cmd> --profiles-dir
/opt/airflow/dbt
```

Data-aware scheduling makes the dependency explicit. dbt starts as soon as the load commits, never before and never on stale data, and a failed ETL run does not trigger dbt at all.

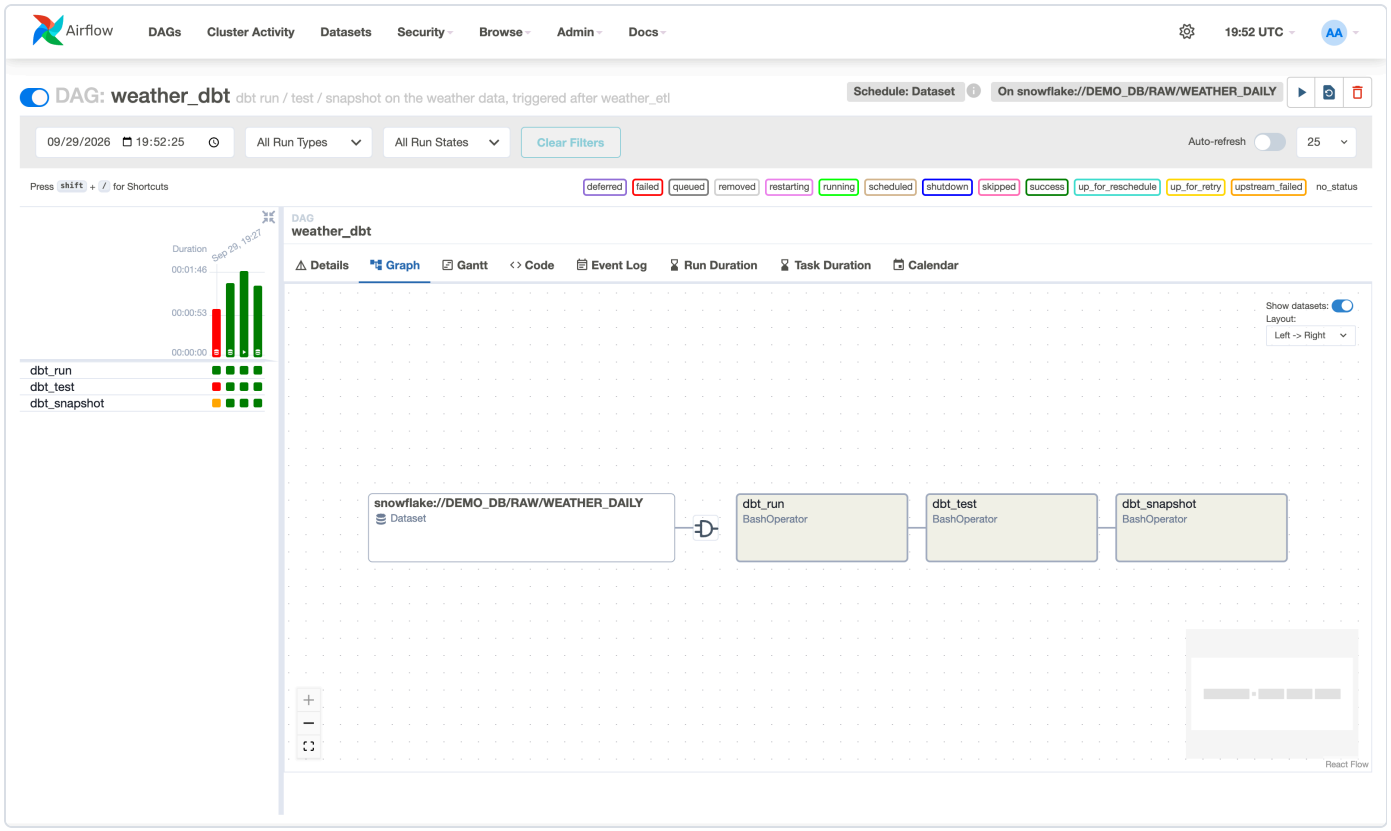


Figure 8 — weather_dbt graph: triggered by the RAW.WEATHER_DAILY Dataset

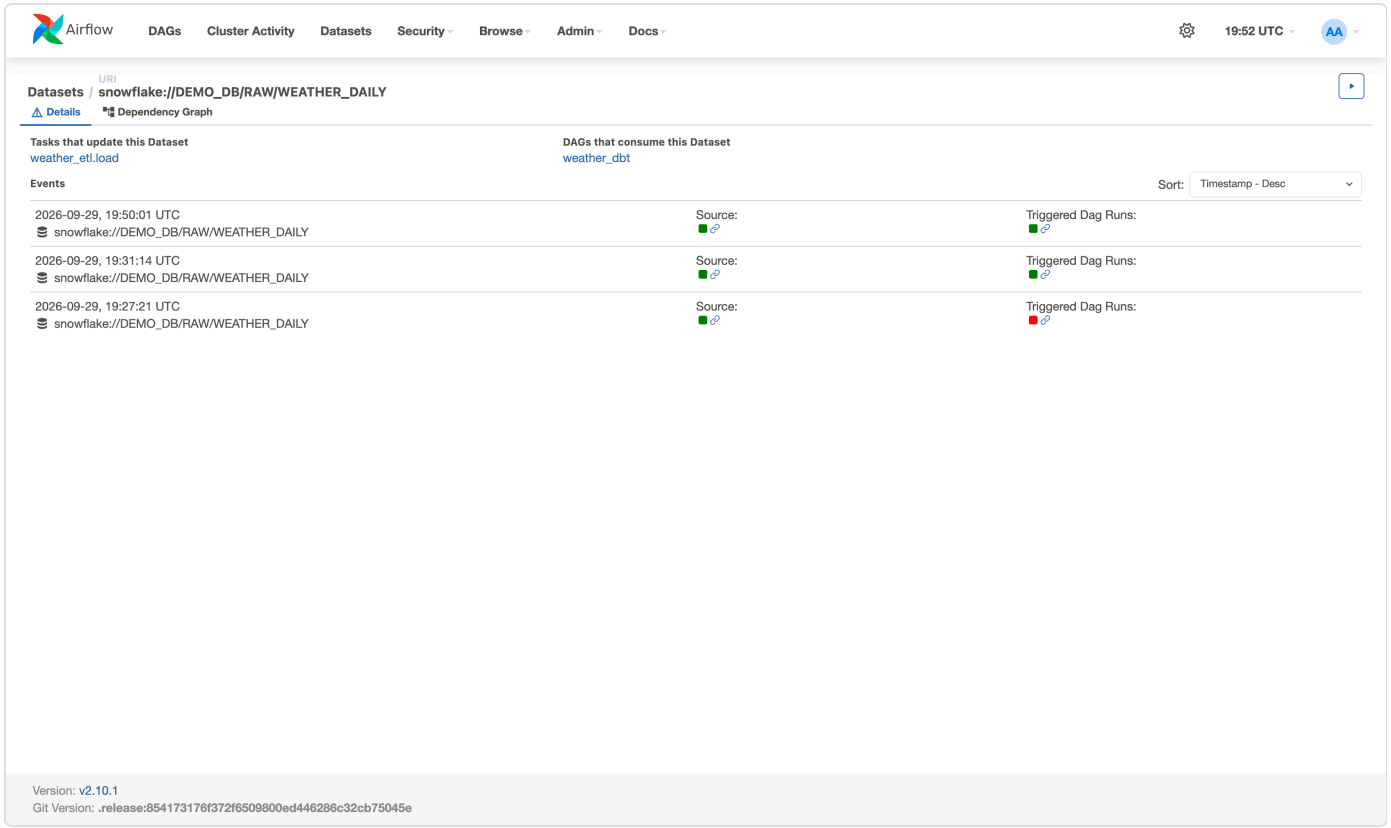


Figure 9 — Dataset page: producer task weather_etl.load, consumer DAG weather_dbt, and the triggered runs

7.5 dbt — models, tests and snapshot

Object	Materialization	Purpose
`stg_weather_daily`	view	Typing, renaming, surrogate key, null handling, WMO code → category
`weather_metrics`	table	Moving averages, anomaly + z-score, rolling rainfall, dry-spell length
`city_weather_summary`	table	Per-city KPIs over observed days
`snapshot_weather_daily`	snapshot (check)	SCD-2 history of revisions to observed values and forecasts
23 data tests	—	unique, not_null, accepted_values, relationships (models + source) and 2 singular tests

Dry-spell length uses a gaps-and-islands window. Every wet day starts a new group, and within a group the consecutive dry days are counted:

```
sum(iff(precipitation_mm < {{ var('dry_day_threshold_mm') }}, 0, 1))
    over (partition by city order by date rows between unbounded preceding and current row) as
wet_day_group
...
iff(is_dry_day,
    sum(iff(is_dry_day, 1, 0)) over (partition by city, wet_day_group order by date
                                     rows between unbounded preceding and current row),
    0) as dry_spell_length_days
```

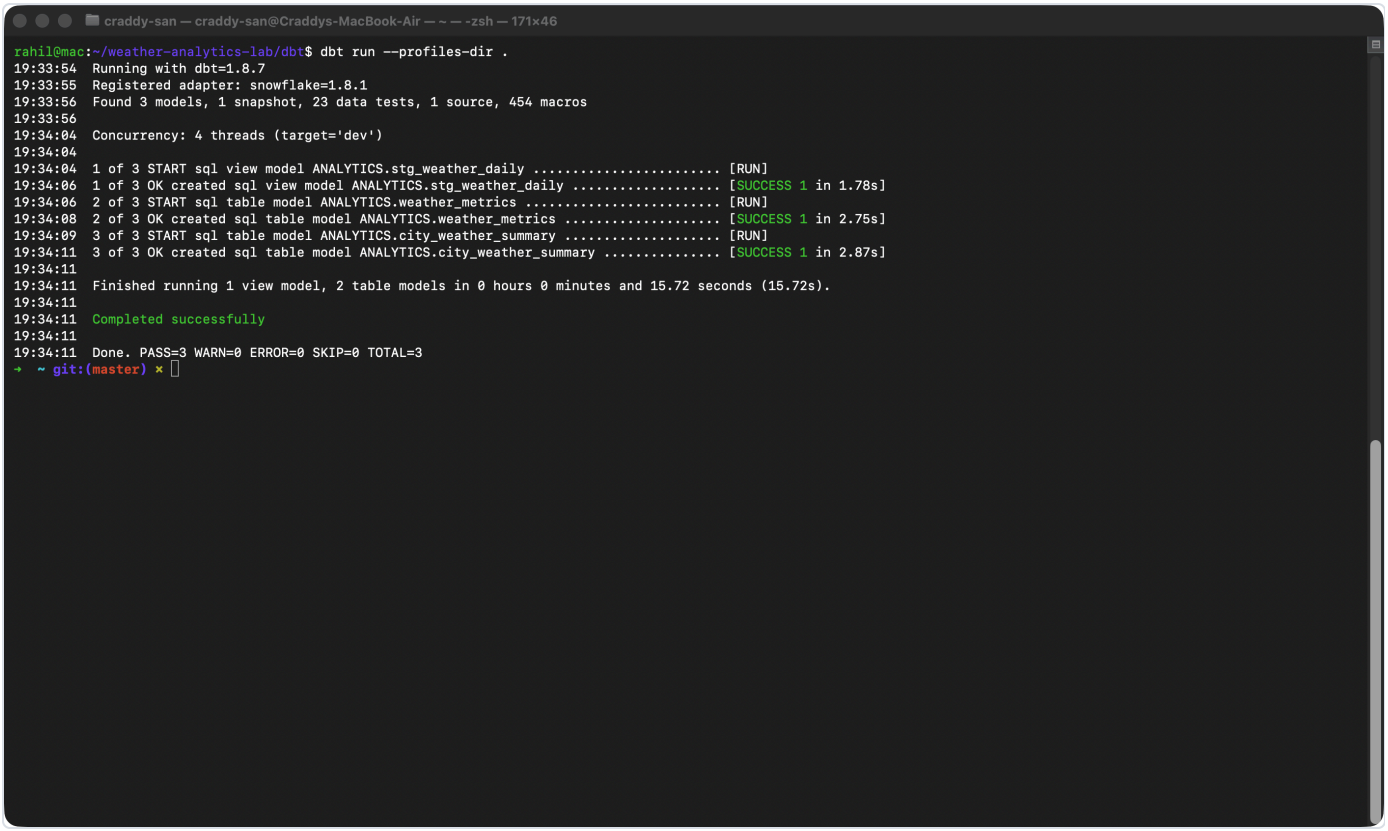


Figure 10 — dbt run: 1 view + 2 tables built

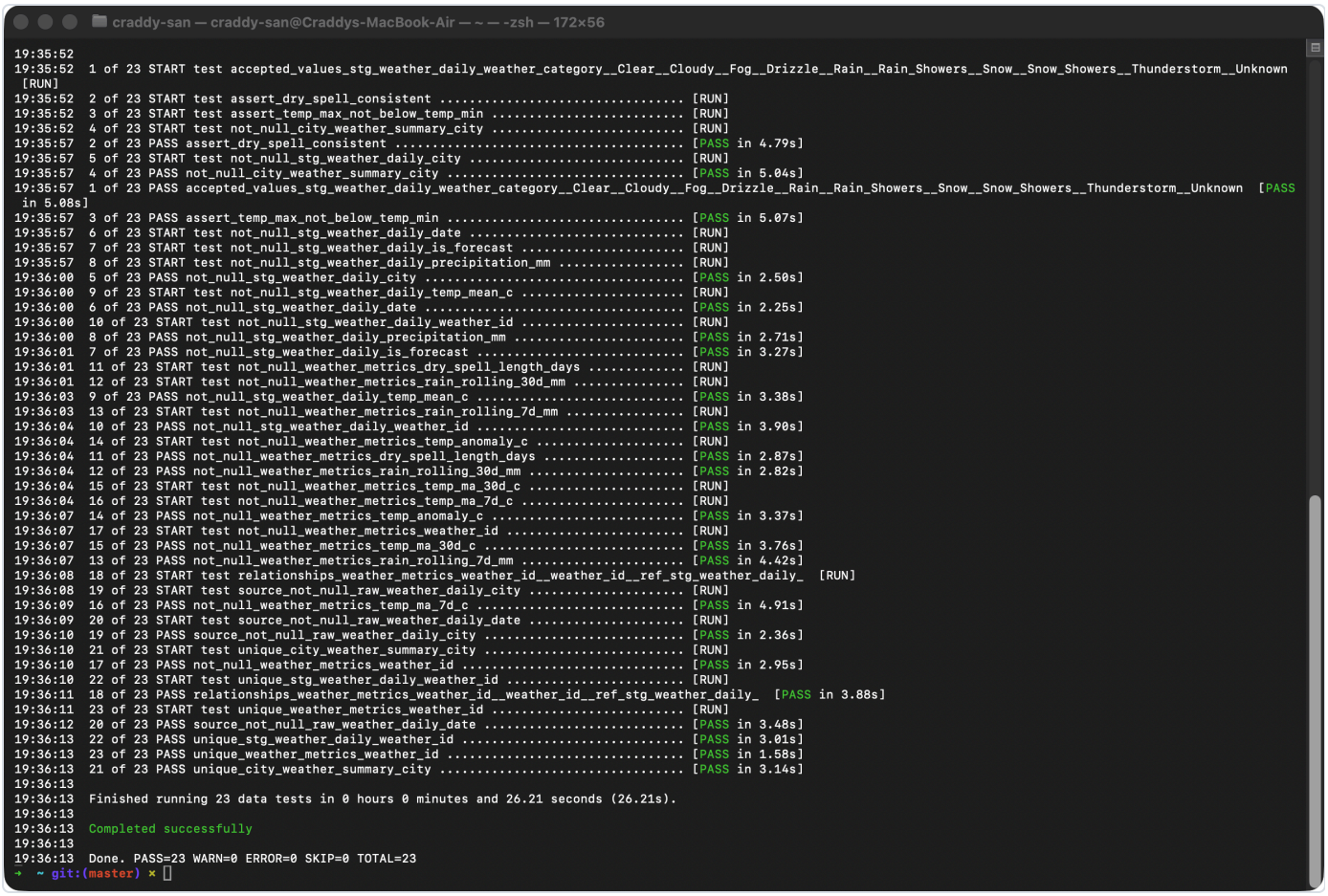


Figure 11 — dbt test: 23 of 23 passed

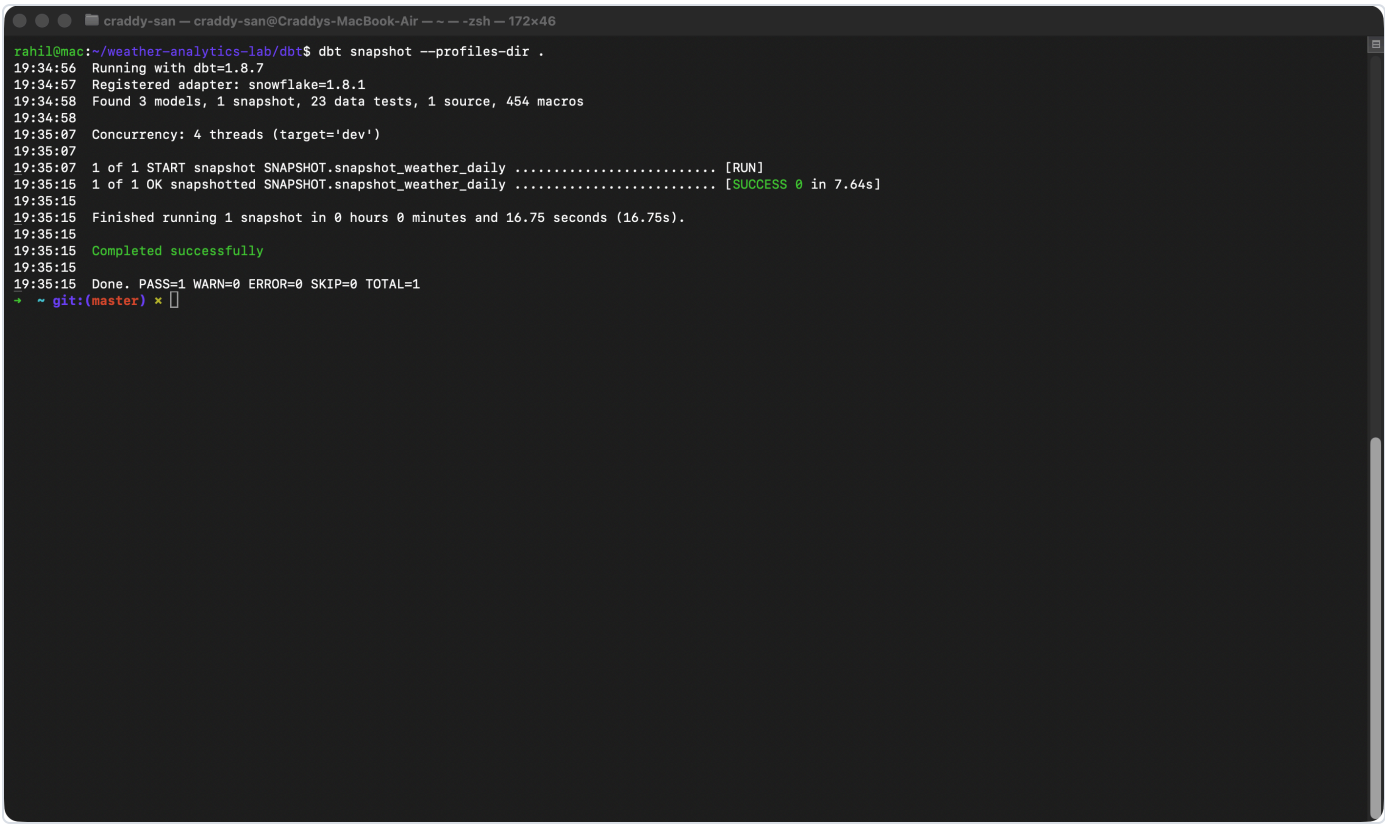
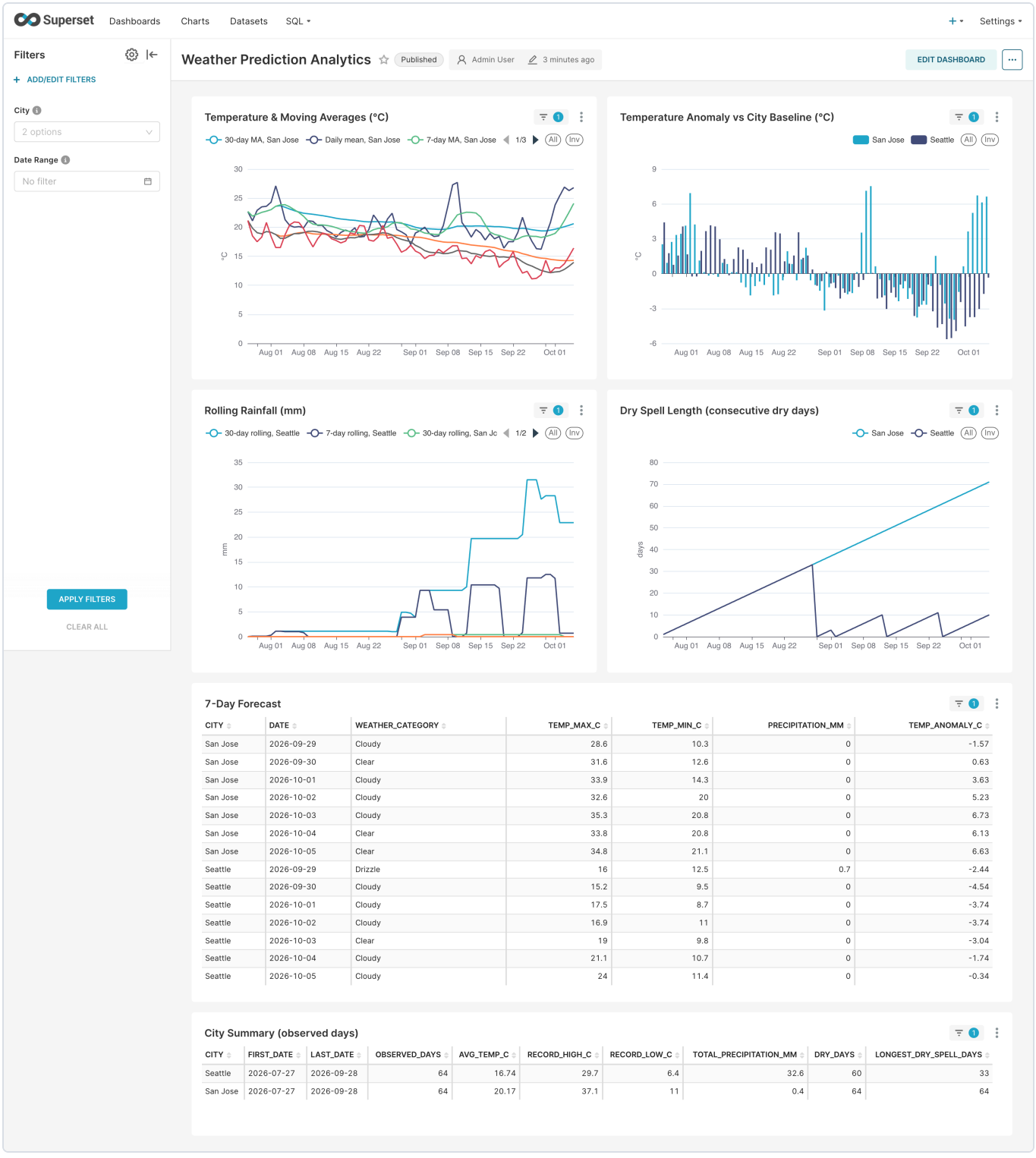


Figure 12 — dbt snapshot

7.6 BI dashboard — Apache Superset

> **Purpose:** compare the recent climate of San Jose and Seattle, and put the 7-day forecast in context: is it warmer or colder than normal, is rain building up, is a dry spell under way?
> **Dataset:** Superset datasets `weather_metrics` (daily metrics per city) and `city_weather_summary` (one row per city), both read live from `DEMO_DB.ANALYTICS` through key-pair auth.
> **Usage:** pick one or more cities in the **City** filter and a period in the **Date Range** filter, then click *Apply filters*. Every chart is in scope of both filters. The whole dashboard can be rebuilt with `superset/bootstrap_dashboard.py` (Superset REST API).

Chart	Type	What it shows
Temperature & Moving Averages	Line	Daily mean vs 7-day and 30-day MA per city
Temperature Anomaly vs City Baseline	Bar	Positive = warmer than the city's normal
Rolling Rainfall	Line	7-day and 30-day rolling precipitation
Dry Spell Length	Line	Consecutive dry days; drops to 0 on a rainy day
7-Day Forecast	Table	Forecast days: category, temperatures, rain and anomaly
City Summary	Table	Average, record high/low, total rain, dry days, longest spell



7-Day Forecast

CITY	DATE	WEATHER_CATEGORY	TEMP_MAX_C	TEMP_MIN_C	PRECIPITATION_MM	TEMP_ANOMALY_C
San Jose	2026-09-29	Cloudy	28.6	10.3	0	-1.57
San Jose	2026-09-30	Clear	31.6	12.6	0	0.63
San Jose	2026-10-01	Cloudy	33.9	14.3	0	3.63
San Jose	2026-10-02	Cloudy	32.6	20	0	5.23
San Jose	2026-10-03	Cloudy	35.3	20.8	0	6.73
San Jose	2026-10-04	Clear	33.8	20.8	0	6.13
San Jose	2026-10-05	Clear	34.8	21.1	0	6.63
Seattle	2026-09-29	Drizzle	16	12.5	0.7	-2.44
Seattle	2026-09-30	Cloudy	15.2	9.5	0	-4.54
Seattle	2026-10-01	Cloudy	17.5	8.7	0	-3.74
Seattle	2026-10-02	Cloudy	16.9	11	0	-3.74
Seattle	2026-10-03	Clear	19	9.8	0	-3.04
Seattle	2026-10-04	Cloudy	21.1	10.7	0	-1.74
Seattle	2026-10-05	Cloudy	24	11.4	0	-0.34

City Summary (observed days)

CITY	FIRST_DATE	LAST_DATE	OBSERVED_DAYS	AVG_TEMP_C	RECORD_HIGH_C	RECORD_LOW_C	TOTAL_PRECIPITATION_MM	DRY_DAYS	LONGEST_DRY_SPELL_DAYS
Seattle	2026-07-27	2026-09-28	64	16.74	29.7	6.4	32.6	60	33
San Jose	2026-07-27	2026-09-28	64	20.17	37.1	11	0.4	64	64

Figure 13 — Full dashboard, no filters (both cities, Jul 27 → Oct 5)

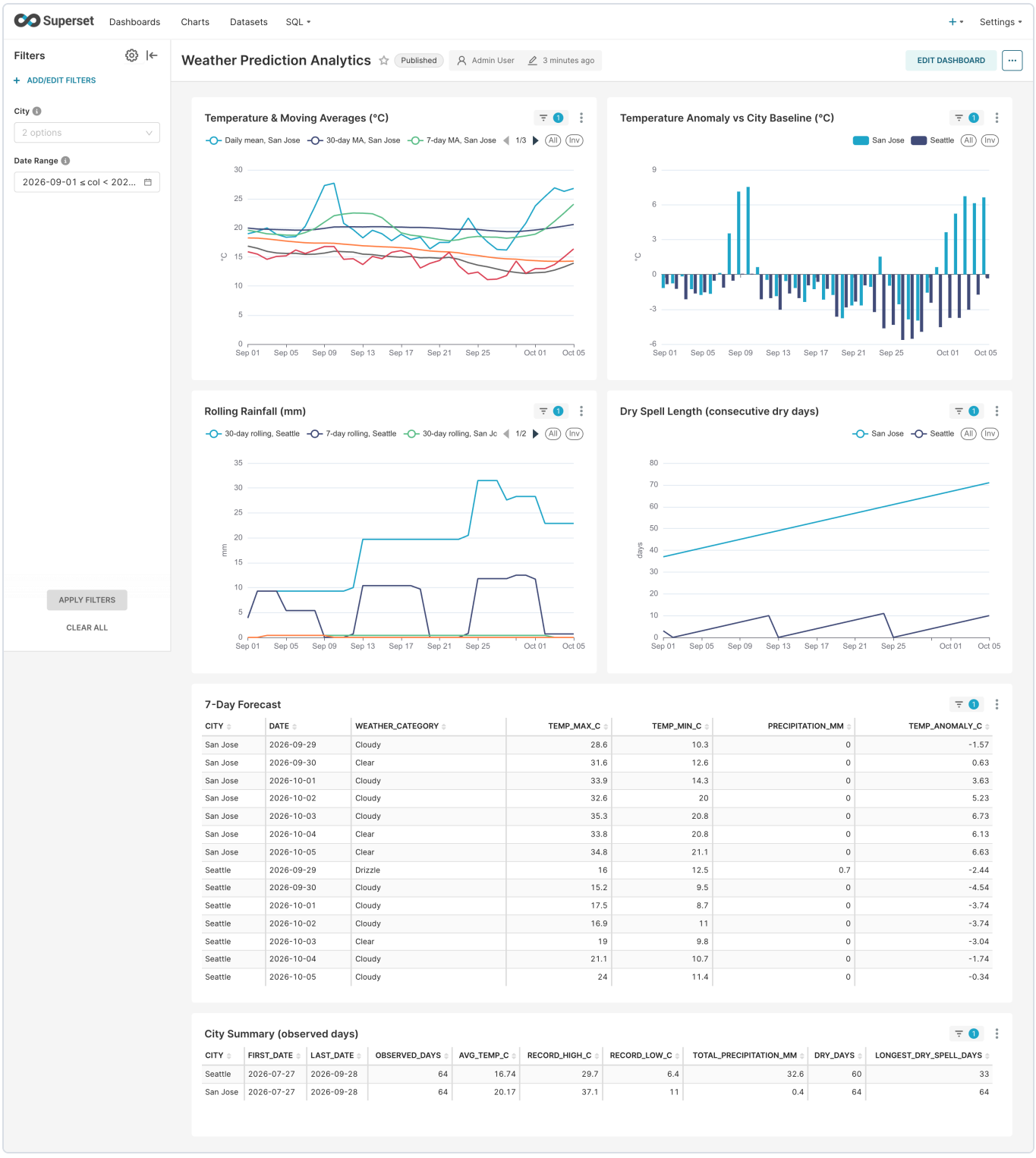


Figure 14 — Date Range changed to 2026-09-01 → 2026-10-06: all time-series charts re-query and the x-axis narrows

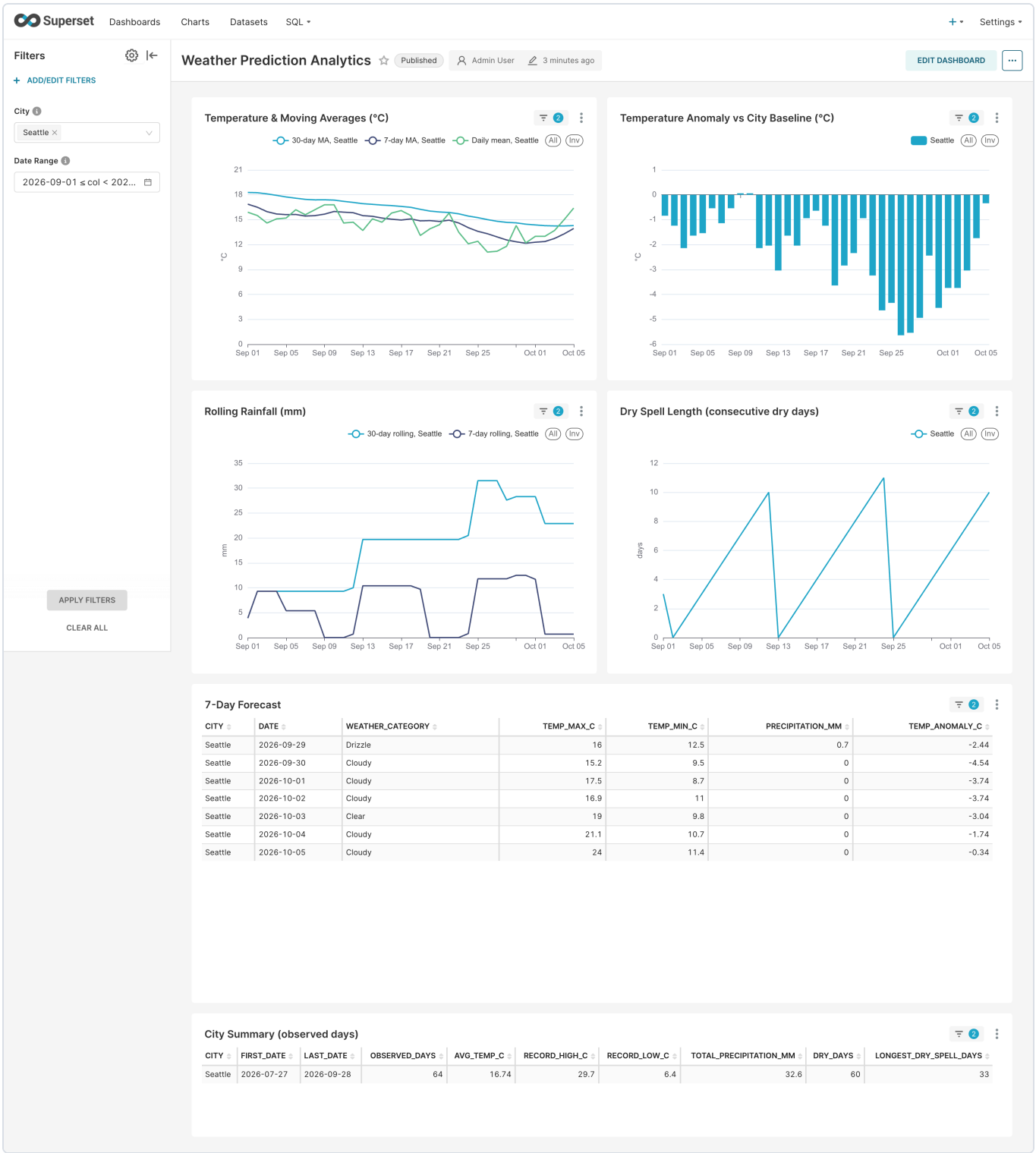


Figure 15 — City filter set to Seattle (with the same date range): only Seattle series and rows remain

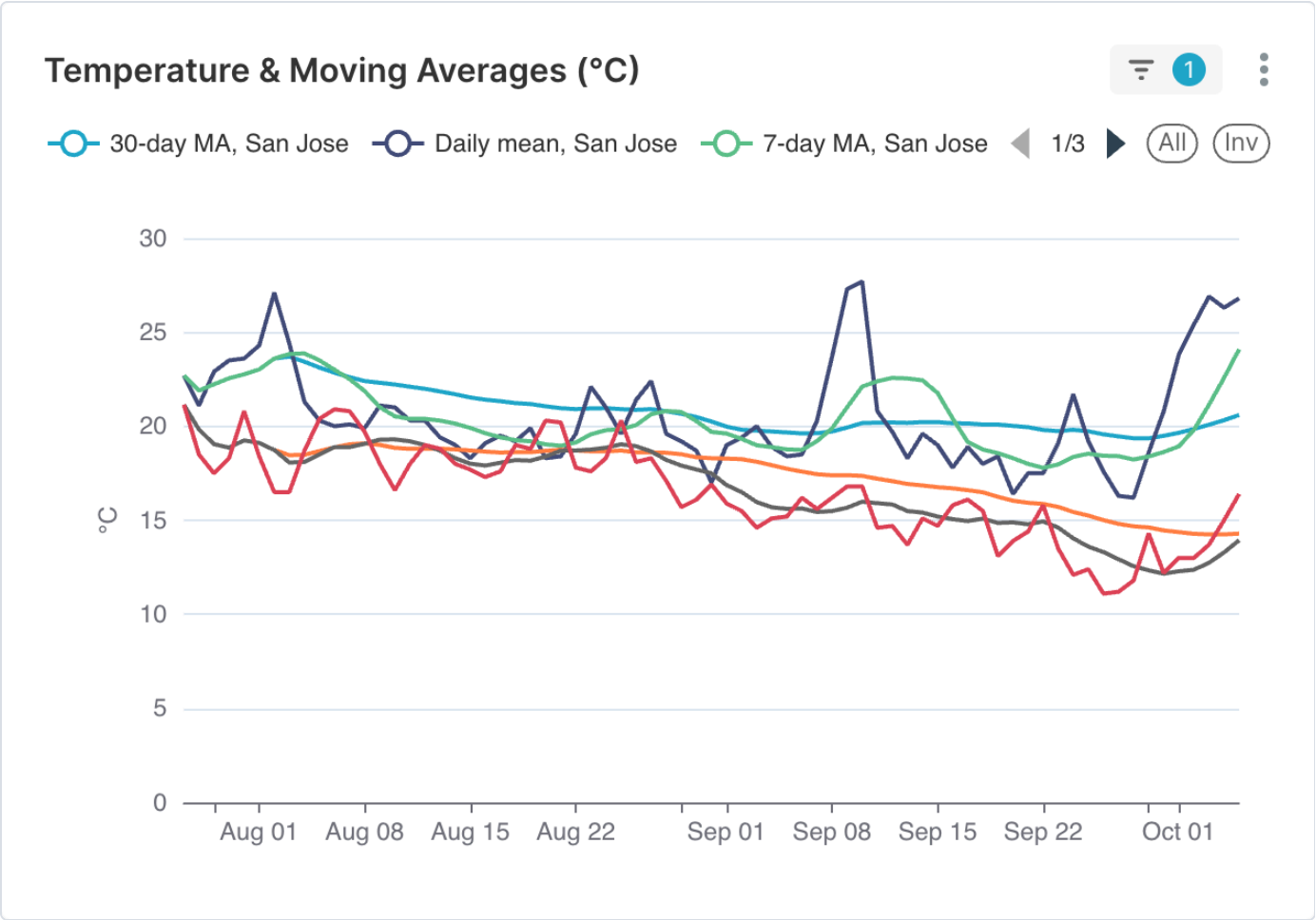


Figure 16 — Temperature & moving averages

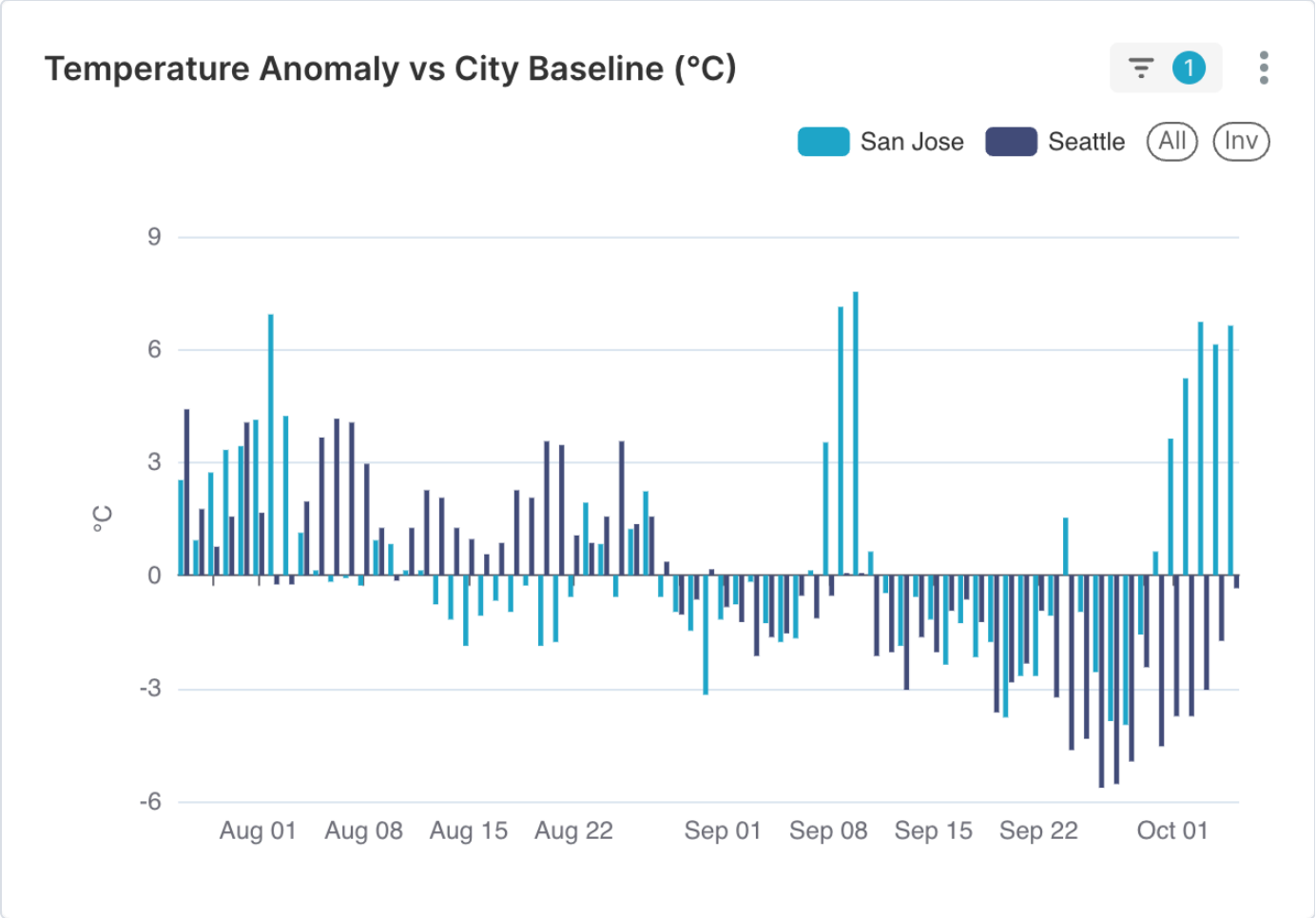


Figure 17 — Temperature anomaly vs city baseline

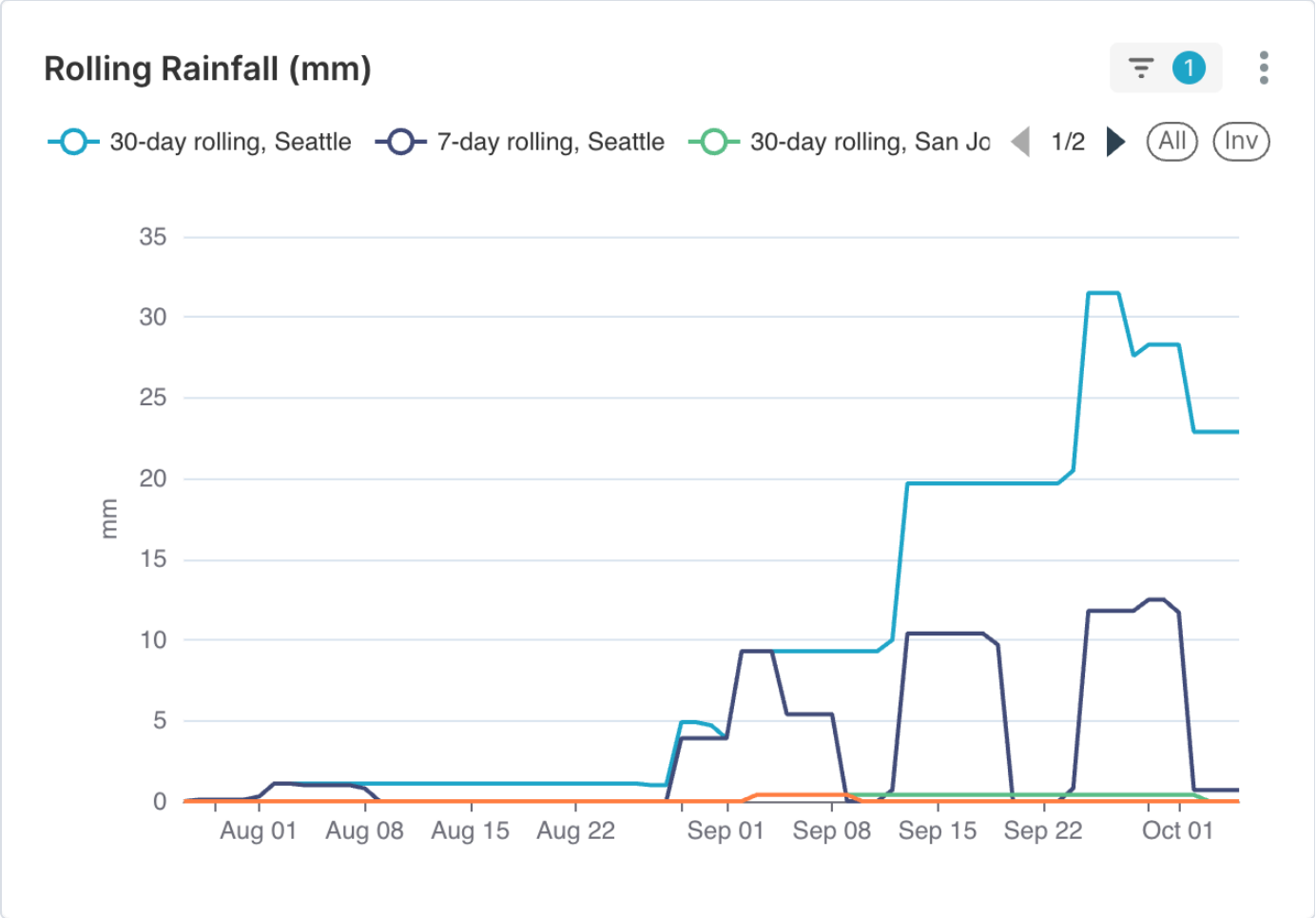


Figure 18 — Rolling rainfall

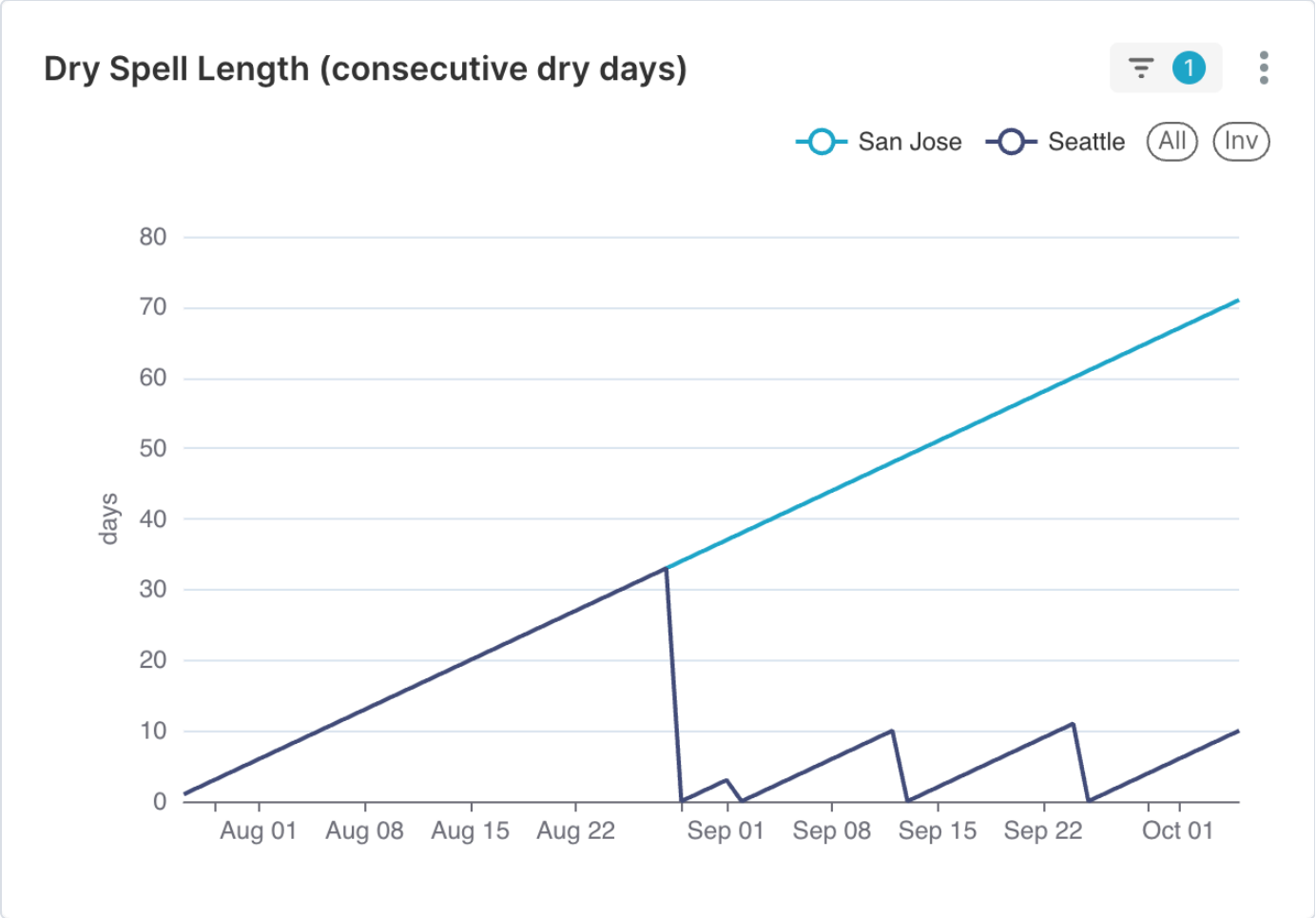


Figure 19 — Dry spell length

7-Day Forecast

CITY	DATE	WEATHER_CATEGORY	TEMP_MAX_C	TEMP_MIN_C	PRECIPITATION_MM	TEMP_ANOMALY_C
San Jose	2026-09-29	Cloudy	28.6	10.3	0	-1.57
San Jose	2026-09-30	Clear	31.6	12.6	0	0.63
San Jose	2026-10-01	Cloudy	33.9	14.3	0	3.63
San Jose	2026-10-02	Cloudy	32.6	20	0	5.23
San Jose	2026-10-03	Cloudy	35.3	20.8	0	6.73
San Jose	2026-10-04	Clear	33.8	20.8	0	6.13
San Jose	2026-10-05	Clear	34.8	21.1	0	6.63
Seattle	2026-09-29	Drizzle	16	12.5	0.7	-2.44
Seattle	2026-09-30	Cloudy	15.2	9.5	0	-4.54
Seattle	2026-10-01	Cloudy	17.5	8.7	0	-3.74
Seattle	2026-10-02	Cloudy	16.9	11	0	-3.74
Seattle	2026-10-03	Clear	19	9.8	0	-3.04
Seattle	2026-10-04	Cloudy	21.1	10.7	0	-1.74
Seattle	2026-10-05	Cloudy	24	11.4	0	-0.34

Figure 20 — 7-day forecast table

City Summary (observed days)

CITY	FIRST_DATE	LAST_DATE	OBSERVED_DAYS	AVG_TEMP_C	RECORD_HIGH_C	RECORD_LOW_C	TOTAL_PRECIPITATION_MM	DRY_DAYS	LONGEST_DRY_SPELL_DAYS
Seattle	2026-07-27	2026-09-28	64	16.74	29.7	6.4	32.6	60	33
San Jose	2026-07-27	2026-09-28	64	20.17	37.1	11	0.4	64	64

Figure 21 — City summary table

7.7 Findings from the dashboard

- **San Jose** had no rain for the whole observed period: a dry spell of 64+ days with only 0.4 mm in total.
- **Seattle**'s rolling 30-day rainfall climbed to about 31 mm through September, and three rain events reset its dry spells.
- Late September in Seattle ran **4–6 °C below** its recent baseline. The San Jose forecast for early October is **5–7 °C above** baseline (forecast highs of 32.6–35.3 °C).

8. Lessons

1. **Data tests pay for themselves immediately.** The first `dbt test` run failed on 56 rows with NULL temperatures. Open-Meteo's forecast endpoint keeps only about 63 days of history, and asking for more returns empty days rather than an error. We fixed the root cause at extraction (skip empty days, fetch 60 days) instead of hiding it downstream.
2. **Idempotency means replacing what you asked for, not only what you received.** Deleting just the returned date range would have left the stale NULL rows behind. The load now deletes the full requested window, so any re-run leaves the table correct.
3. **In Snowflake, DDL auto-commits.** `CREATE TABLE` inside a `BEGIN … COMMIT` block silently ends the transaction, so the DDL has to run before `BEGIN`.
4. **Dataset scheduling beats time-offset crons.** Scheduling dbt "30 minutes after the ETL" breaks whenever the ETL runs long or fails. A Dataset dependency runs dbt exactly when new data is committed.
5. **Configuration over code pays off.** Our original Snowflake trial expired mid-project. Because every credential was in the Airflow Connection and dbt profile environment variables, moving to a new account took one connection update and no code changes.
6. **Identifier case matters across tools.** Snowflake reports unquoted identifiers in UPPER case, so Superset datasets expose `CITY` and `DATE`, not `city`. Charts built with lower-case column names fail with "columns missing".
7. **Key-pair authentication** avoids password/MFA prompts for service connections, and the same private key file works in Airflow, dbt and Superset. The key stays out of git; only its path is configured.

9. Future Work

- **Forecast accuracy scoring:** the snapshot already stores each forecast version. Joining the version captured N days ahead to the final observed row gives forecast error (MAE) by lead time and city.
- **Climate-normal baselines:** backfill years of data from Open-Meteo's Historical Weather (ERA5) API, so anomalies are measured against a 30-year normal instead of the recent window.
- **Incremental models + freshness:** switch `weather_metrics` to an incremental merge and add `dbt source freshness` as a DAG task with alerting.
- **More cities and variables:** humidity, UV index, and heat-index or frost-risk alerts (cities are configuration only).
- **Operations and security:** Slack/email alerts on failure, CI that runs `dbt build` on pull requests, a secrets backend instead of the Airflow metadata DB, and a least-privilege Snowflake role instead of ACCOUNTADMIN.

10. Conclusion

We delivered a fully automated daily ELT pipeline:

- **Airflow** extracts weather for two cities in parallel and loads Snowflake idempotently inside one SQL transaction. Credentials and settings come from Airflow Connections and Variables.
- **dbt** is triggered by an Airflow Dataset as soon as new data lands. It builds tested metric models (23 of 23 tests passing) and keeps an SCD-2 snapshot of forecast revisions.
- **Superset** turns the metrics into an interactive dashboard with City and Date Range filters.

The results clearly separate the two climates: San Jose's dry spell lasted more than two months, while Seattle's September rain shows up in its rolling rainfall. The anomaly view shows at a glance that the upcoming San Jose forecast is well above normal. Every stage can be safely re-run, is version-controlled in the repository, and is extended through configuration.

> **GitHub repository:** https://github.com/rahil88/weather-analytics-lab

11. References

1. Open-Meteo, Weather Forecast API documentation. https://open-meteo.com/en/docs
2. WMO weather interpretation codes (via Open-Meteo docs). https://open-meteo.com/en/docs#weathervariables
3. Apache Airflow 2.10: Data-aware scheduling (Datasets). https://airflow.apache.org/docs/apache-airflow/2.10.1/authoring-and-scheduling/datasets.html
4. Apache Airflow 2.10: Dynamic task mapping. https://airflow.apache.org/docs/apache-airflow/2.10.1/authoring-and-scheduling/dynamic-task-mapping.html
5. Apache Airflow Snowflake provider. https://airflow.apache.org/docs/apache-airflow-providers-snowflake/stable/
6. Snowflake, Transactions. https://docs.snowflake.com/en/sql-reference/transactions
7. Snowflake, Key-pair authentication. https://docs.snowflake.com/en/user-guide/key-pair-auth
8. dbt, Snapshots. https://docs.getdbt.com/docs/build/snapshots
9. dbt, Data tests. https://docs.getdbt.com/docs/build/data-tests
10. dbt, Snowflake setup. https://docs.getdbt.com/docs/core/connect-data-platform/snowflake-setup
11. Apache Superset, REST API. https://superset.apache.org/docs/api
12. Apache Superset, Connecting to Snowflake. https://superset.apache.org/docs/configuration/databases#snowflake